

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«Национальный исследовательский университет ИТМО»
Факультет Программной инженерии и компьютерной техники

Лабораторная работа №2
по дисциплине «Архитектура программных систем»

Преподаватель:
Перл Иван Андреевич

Выполнил:
Антипин Григорий Викторович
Группа: Р3332

СОДЕРЖАНИЕ

ЗАДАНИЕ.....	3
Singleton (GoF)	4
Описание.....	4
Сценарии использования	4
Общие ограничения.....	5
Factory Method (GoF)	5
Описание.....	5
Сценарии использования	5
Общие ограничения.....	6
Controller (GRASP)	7
Описание.....	7
Сценарии использования	7
Общие ограничения.....	8
Information Expert (GRASP).....	9
Описание.....	9
Сценарии использования	9
Общие ограничения.....	10
ВЫВОД	10

ЗАДАНИЕ

Из списка шаблонов проектирования GoF и GRASP выбрать 3-4 шаблона и для каждого из них придумать 2-3 сценария, для решения которых могут применены выбранные шаблоны. Сделать предположение о возможных ограничениях, к которым можем привести использование шаблона в каждом описанном случае. Обязательно выбрать шаблоны из обоих списков.

Singleton (GoF)

Описание

Singleton - порождающий шаблон проектирования из группы GoF.

Он гарантирует, что у класса существует только один экземпляр, и предоставляет глобальную точку доступа к этому экземпляру. Чаще всего применяется для объектов, которые должны централизованно управлять общими ресурсами (логирование, конфигурация, подключение к сервису).

Сценарии использования

1. Единый логгер приложения

В большом приложении (backend + фоновые задачи) все компоненты пишут в общий журнал. Singleton гарантирует, что логирование идёт через один объект, который управляет форматом, уровнем логов и выводом (файл/консоль/удалённый сервер).

Ограничения:

- Сложно сделать разные логгеры для разных подсистем (например, отдельный лог для платежей).
- В тестах тяжело изолировать вывод (логгер «глобальный», его надо подменять аккуратно).

2. Централизованный доступ к конфигурации

Config хранит параметры (URL сервисов, таймауты, флаги функциональности). Singleton даёт единый источник правды, чтобы модули не таскали конфиг по всем методам.

Ограничения:

- Подмена конфигурации в тестах усложняется: нужен сброс состояния Singleton
- Может превратиться в «свалку» настроек без структуры.

3. Менеджер кэша

Например, CacheManager управляет in-memory кэшем (LRU/TTL), чтобы не плодить разные кэши в разных местах. Singleton позволяет согласованно очищать/обновлять кэш и считать статистику (hits/misses).

Ограничения:

- Если приложение многопоточное, нужны блокировки/потокобезопасность → падение производительности.
- Глобальный кэш может стать узким местом при росте нагрузки

Общие ограничения

1. Скрытая глобальная зависимость - код использует Singleton неявно, сложнее анализировать зависимости.
2. Проблемы тестирования - общий объект сохраняет состояние между тестами, нужна очистка/моки.
3. Риск превращения в “God Object” - в Singleton начинают складывать “всё подряд”.

Factory Method (GoF)

Описание

Factory Method (Фабричный метод) — порождающий шаблон из группы GoF.

Он определяет интерфейс для создания объектов, но позволяет подклассам (или конкретным фабрикам) решать, какой класс создавать. Шаблон нужен, когда код должен работать с абстракциями и не зависеть от конкретных типов создаваемых объектов.

Сценарии использования

1. Создание уведомлений разными каналами

Система отправляет уведомления: Email/SMS/Push. В зависимости от настроек пользователя выбирается нужный “создатель”, который создаёт конкретный объект уведомления.

Ограничения:

- При росте каналов растёт число классов (каждый новый канал = новый тип + фабрика/подкласс).
- Логика выбора фабрики может «размазаться» по коду, если не централизовать.

2. Создание парсеров для разных форматов

Есть импортер, который принимает файл и создаёт нужный парсер: CSV/JSON/XML. Factory Method позволяет добавлять форматы без переписывания импортера.

Ограничения:

- Ошибки определения типа файла (неправильное расширение/контент) требуют доп. логики.
- Сложнее дебажить цепочку: “кто именно создал вот этот парсер”.

3. Создание подключений к разным источникам данных

Приложение работает то с PostgreSQL, то с SQLite, то с REST API как “источником данных”. Фабрика создаёт нужный DataSource и возвращает общий интерфейс.

Ограничения:

- Разные источники часто имеют разные возможности → интерфейс либо становится слишком общим, либо ломается LSP.
- Возможна избыточная абстракция, если в проекте реально используется один источник.

Общие ограничения

1. Рост количества классов - добавление новых продуктов увеличивает число файлов/типов
2. Сложность навигации - “где создаётся объект” становится неочевидно
3. Over-engineering - если вариантов создания мало и почти не меняются, шаблон может быть лишним.

Controller (GRASP)

Описание

Controller (Контроллер) — шаблон из GRASP, который рекомендует назначить объект, отвечающий за приём внешних запросов (UI, API, события) и координацию действий системы. Контроллер не должен содержать всю бизнес-логику, его задача — принять запрос, вызвать нужные сущности/сервисы и вернуть результат.

Сценарии использования

1. Контроллер заказа в интернет-магазине

OrderController принимает запрос “Оформить заказ”, проверяет входные данные, вызывает расчёт цены, списание, создание заказа, отправку уведомления

Ограничения:

- Контроллер легко превращается в “толстый” класс, если туда запихнуть бизнес-логику
- При большом числе операций контроллер становится трудно тестировать (много зависимостей).

2. Контроллер авторизации

AuthController обрабатывает вход/выход, обновление токенов, выдачу сессий, регистрацию. Он связывает UI/API с сервисами безопасности

Ограничения:

- Риск смешивания ответственности (валидация, безопасность, бизнес-правила) в одном месте.
- Ошибки в контроллере влияют на всю систему входа (критический компонент).

3. Контроллер управления задачами

TaskController принимает команды: создать/изменить/закрыть задачу, назначить исполнителя, добавить комментарий. Он управляет транзакциями и вызывает доменные сущности.

Ограничения:

- При расширении функционала контроллер разрастается быстрее всего.
- Если контроллер “знает всё”, появляется сильная связанность с доменной моделью.

Общие ограничения

1. Риск когда в контроллере оказывается вся логика приложения
2. Высокая связанность - контроллер часто становится центром зависимостей
3. Сложность тестирования - много моков/стабов из-за большого числа вызовов.

Information Expert (GRASP)

Описание

Information Expert (Информационный эксперт) — шаблон GRASP, который говорит: “Назначай ответственность тому классу, который имеет нужные данные для её выполнения.”

Идея в том, что вычисления и решения должны находиться рядом с информацией, которую они используют — тогда система получается логичнее и менее связной.

Сценарии использования

1. Корзина сама считает итоговую сумму

Cart хранит позиции и умеет считать total (с учётом количества, цены, скидок). Так логика не размазывается по UI или контроллеру.

Ограничения:

- Корзина может “распухнуть”, если туда добавить ещё и акции/купоны/доставку.
- Если правила расчёта часто меняются, придётся часто менять класс Cart.

2. Заказ сам определяет свой статус

Order содержит данные (оплата, сборка, доставка) и сам вычисляет статус: NEW/PAID/SHIPPED/DELIVERED.

Ограничения:

- При сложном жизненном цикле статус превращается в сложную логику внутри Order.
- Возможны циклические зависимости, если статус зависит от внешних сервисов.

3. Склад сам проверяет доступность товаров

Inventory хранит остатки и сам отвечает на вопрос “Можно ли собрать заказ?”. Это логичнее, чем делать расчёт в контроллере.

Ограничения:

- В реальности остатки могут быть в БД/внешней системе — класс становится “экспертом” без реальных данных.
- Проблемы конкурентности (два заказа одновременно резервируют один товар).

Общие ограничения

1. Разрастание классов-экспертов - “эксперт” может стать перегруженным ответственностями.
2. Сложность при внешних данных - если данные не внутри объекта (микросервисы/БД), принцип сложнее соблюдать.
3. Частые изменения правил - если правила вычислений постоянно меняются, классы придётся часто модифицировать.

ВЫВОД

В ходе выполнения лабораторной работы были рассмотрены и проанализированы основные шаблоны проектирования из групп GoF и GRASP: Singleton, Factory Method, Controller и Information Expert. Для каждого шаблона было изучено его назначение, принципы применения и приведены практические сценарии использования в программных системах различного назначения.

В результате выполнения работы было показано, что применение шаблонов проектирования позволяет повысить гибкость архитектуры, снизить связанность компонентов, упростить расширение функциональности и улучшить читаемость кода. Вместе с тем было выявлено, что неумеренное или необоснованное использование шаблонов может приводить к усложнению структуры проекта, росту количества классов и возникновению избыточных абстракций.

Таким образом, можно сделать вывод, что шаблоны проектирования являются эффективным инструментом построения устойчивых и расширяемых программных систем, однако их применение должно быть осознанным и соответствовать реальным потребностям проекта, а не использоваться формально или избыточно.